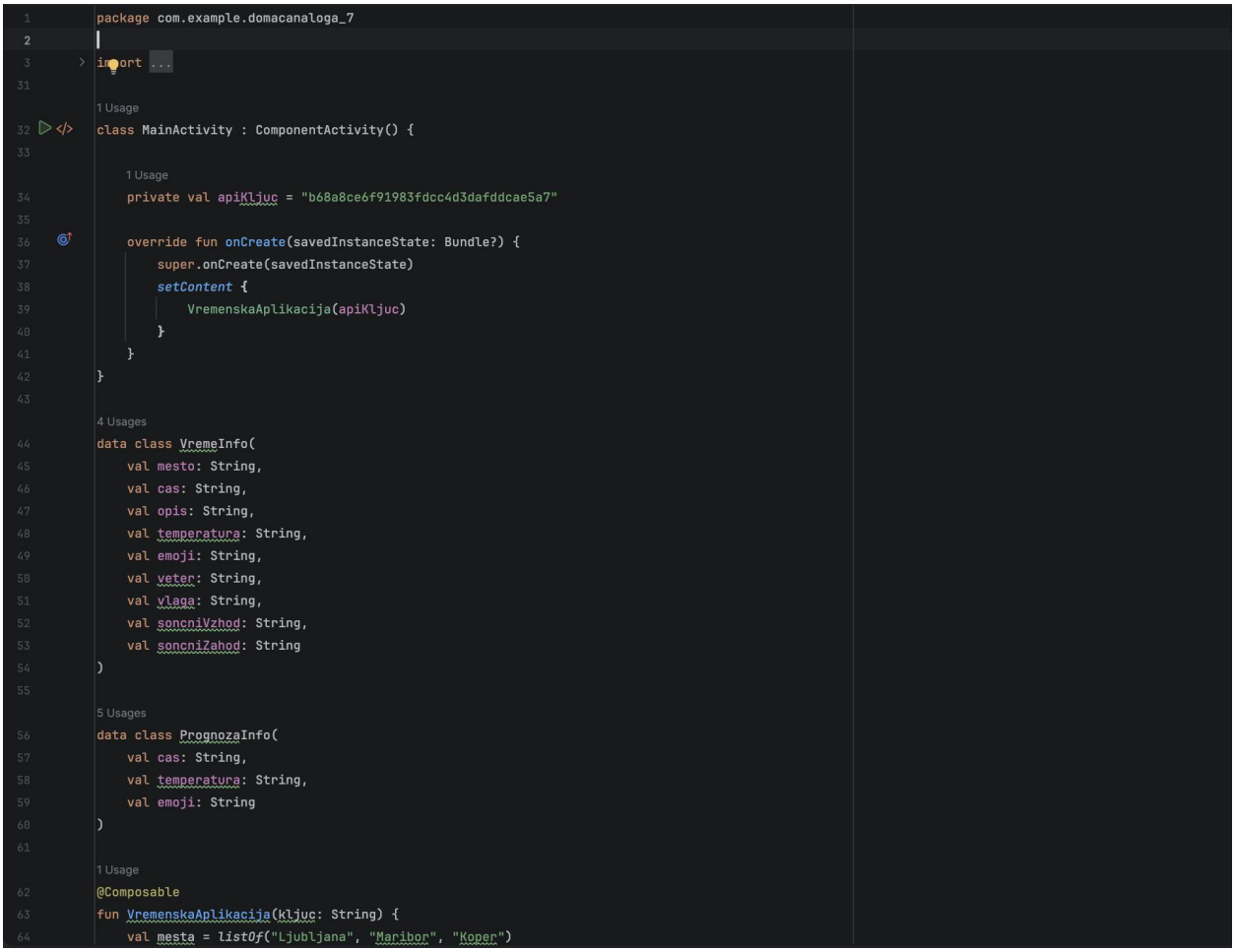
Odprta koda – Domača naloga 7

Marko Tojagić

7_KODA



```
package com.example.domacanaloga_7

import ...

1 Usage
class MainActivity : ComponentActivity() {

    1 Usage
    private val apiKljuc = "b68a8ce6f91983fdcc4d3dafddcae5a7"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            VremenskaAplikacija(apiKljuc)
        }
    }
}

4 Usages
data class VremeInfo(
    val mesto: String,
    val cas: String,
    val opis: String,
    val temperatura: String,
    val emoji: String,
    val veter: String,
    val vlaga: String,
    val soncniVzhod: String,
    val soncniZahod: String
)

5 Usages
data class PrognozaInfo(
    val cas: String,
    val temperatura: String,
    val emoji: String
)

1 Usage
@Composable
fun VremenskaAplikacija(kljuc: String) {
    val mesta = listOf("Ljubljana", "Maribor", "Koper")
```

```
65
66      var izbranoMesto by remember { mutableStateOf( value = mesta[0]) }
67      var razsirjeno by remember { mutableStateOf( value = false) }
68      var vreme by remember { mutableStateOf<VremeInfo?>( value = null) }
69      var prognoze by remember { mutableStateOf<List<PrognozaInfo>>( value = emptyList()) }
70      var napaka by remember { mutableStateOf( value = "") }
71
72      LaunchedEffect( key1 = izbranoMesto) {
73          try {
74              val (trenutno, seznamPrognoz) = pridobiVremeInPrognozo(izbranoMesto, kljuc)
75              vreme = trenutno
76              prognoze = seznamPrognoz
77              napaka = ""
78          } catch (_: Exception) {
79              napaka = "Nekaj je šlo narobe pri vremenu."
80          }
81      }
82
83      val gradient = Brush.linearGradient(
84          colors = listOf(Color(0xFF1A2A3A), Color(0xFF0F1A24)),
85          start = Offset( x = 0f,  y = 0f),
86          end = Offset( x = 0f,  y = Float.POSITIVE_INFINITY)
87      )
88
89      Column(
90          modifier = Modifier
91              .fillMaxSize()
92              .background( brush = gradient)
93              .padding( all = 20.dp),
94          horizontalAlignment = Alignment.CenterHorizontally
95      ) {
96          Spacer(modifier = Modifier.height(30.dp))
97
98          Text(
99              text = "🌤️ Vreme",
100             color = Color.White,
101             fontSize = 34.sp,
102             fontWeight = FontWeight.Light,
103             letterSpacing = 2.sp,
104             modifier = Modifier.shadow( elevation = 4.dp,  shape = RoundedCornerShape( size = 8.dp))
```

```
    )

    Spacer(modifier = Modifier.height(24.dp))

    Box {
        OutlinedButton(
            onClick = { razsirjeno = true },
            shape = RoundedCornerShape( size = 30.dp),
            colors = ButtonDefaults.outlinedButtonColors(
                contentColor = Color.White,
                containerColor = Color.Transparent
            )
        ) {
            Text(izbranoMesto, fontSize = 18.sp)
        }

        DropdownMenu(
            expanded = razsirjeno,
            onDismissRequest = { razsirjeno = false },
            modifier = Modifier.background(Color(0xFF2C3E50))
        ) {
            mesta.forEach { m ->
                DropdownMenuItem(
                    text = { Text(m, color = Color.White) },
                    onClick = {
                        izbranoMesto = m
                        razsirjeno = false
                    },
                    modifier = Modifier.background(Color.Transparent)
                )
            }
        }
    }

    Spacer(modifier = Modifier.height(30.dp))

    if (napaka.isNotEmpty()) {
        Card(
```

```
141             if (napaka.isNotEmpty()) {
142                 Card(
143                     colors = CardDefaults.cardColors(containerColor = Color(0x99FF5555)),
144                     shape = RoundedCornerShape( size = 16.dp),
145                     modifier = Modifier.padding( all = 8.dp)
146                 ) {
147                     Text(napaka, color = Color.White, modifier = Modifier.padding( all = 16.dp))
148                 }
149             } else if (vreme == null) {
150                 CircularProgressIndicator(color = Color(0xFF64B5F6))
151             } else {
152                 Vreme( vremePodatki = vreme!!, prognoze)
153             }
154         }
155     }
156
        1 Usage
157     @Composable
158     fun Vreme(vremePodatki: VremeInfo, prognoze: List<PrognozaInfo>) {
159         Column(
160             modifier = Modifier
161                 .fillMaxWidth()
162                 .padding(horizontal = 8.dp)
163                 .verticalScroll( state = rememberScrollState()),
164             horizontalAlignment = Alignment.CenterHorizontally
165         ) {
166             Card(
167                 modifier = Modifier
168                     .fillMaxWidth()
169                     .padding(vertical = 8.dp)
170                     .shadow( elevation = 12.dp,  shape = RoundedCornerShape( size = 32.dp)),
171                 shape = RoundedCornerShape( size = 32.dp),
172                 colors = CardDefaults.cardColors(
173                     containerColor = Color(0xAA1E2A36)
174                 )
175             ) {
176                 Column(
177                     modifier = Modifier.padding( all = 24.dp),
178                     horizontalAlignment = Alignment.CenterHorizontally
```

```
            colors = CardDefaults.cardColors(
                containerColor = Color(0xAA1E2A36)
            )
        ) {
            Column(
                modifier = Modifier.padding( all = 24.dp),
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                Text(
                    "${vremePodatki.mesto}, ${vremePodatki.cas}",
                    color = Color(0xFFB0D4FF),
                    fontSize = 16.sp,
                    letterSpacing = 0.5.sp
                )

                Spacer(modifier = Modifier.height(16.dp))

                Text(
                    text = vremePodatki.opis.replaceFirstChar { it.uppercase() },
                    color = Color.White,
                    fontSize = 24.sp,
                    fontWeight = FontWeight.Medium,
                    textAlign = TextAlign.Center
                )

                Spacer(modifier = Modifier.height(20.dp))

                Row(
                    verticalAlignment = Alignment.CenterVertically,
                    horizontalArrangement = Arrangement.Center,
                    modifier = Modifier.fillMaxWidth()
                ) {
                    Text(
                        text = vremePodatki.temperatura,
                        color = Color.White,
                        fontSize = 68.sp,
                        fontWeight = FontWeight.Bold,
                        letterSpacing = 2.sp
                    )
```

```
                    Spacer(modifier = Modifier.width(20.dp))

                    Box(
                        modifier = Modifier
                            .size(100.dp)
                            .clip( shape = RoundedCornerShape( size = 30.dp))
                            .background(Color.White.copy(alpha = 0.15f)),
                        contentAlignment = Alignment.Center
                    ) {
                        Text(vremePodatki.emoji, fontSize = 54.sp)
                    }
                }
            }
        }

        Spacer(modifier = Modifier.height(20.dp))

        Card(
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape( size = 24.dp),
            colors = CardDefaults.cardColors(containerColor = Color(0xCC1E2A36))
        ) {
            Column(modifier = Modifier.padding( all = 20.dp)) {
                InfoVrstica( oznaka = "💨 Hitrost vetra",  vrednost = vremePodatki.veter)
                InfoVrstica( oznaka = "💧 Vlažnost",  vrednost = vremePodatki.vlaga)
                Divider(color = Color.White.copy(alpha = 0.2f), modifier = Modifier.padding(vertical = 8.dp))
                InfoVrstica( oznaka = "🌅 Sončni vzhod",  vrednost = vremePodatki.soncniVzhod)
                InfoVrstica( oznaka = "🌇 Sončni zahod",  vrednost = vremePodatki.soncniZahod)
            }
        }

        Spacer(modifier = Modifier.height(24.dp))

        Text(
            text = "📆 Napoved",
            color = Color.White,
```

```
            fontSize = 18.sp,
            fontWeight = FontWeight.SemiBold,
            modifier = Modifier.padding(bottom = 12.dp)
        )

        if (prognoze.size >= 3) {
            NapovedVrstica( cas = "Čez 1 uro",  temp = prognoze[0].temperatura, prognoze[0].emoji)
            NapovedVrstica( cas = "Čez 2 uri",  temp = prognoze[1].temperatura, prognoze[1].emoji)
            NapovedVrstica( cas = "Čez 3 ure",  temp = prognoze[2].temperatura, prognoze[2].emoji)
        } else {
            NapovedVrstica( cas = "Čez 1 uro",  temp = vremePodatki.temperatura, vremePodatki.emoji)
            NapovedVrstica( cas = "Čez 2 uri",  temp = vremePodatki.temperatura, vremePodatki.emoji)
            NapovedVrstica( cas = "Čez 3 ure",  temp = vremePodatki.temperatura, vremePodatki.emoji)
        }

        Spacer(modifier = Modifier.height(20.dp))
    }
}

4 Usages
@Composable
fun InfoVrstica(oznaka: String, vrednost: String) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 4.dp),
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
        Text(oznaka, color = Color(0xFFB8D0E8), fontSize = 15.sp)
        Text(vrednost, color = Color.White, fontWeight = FontWeight.Medium, fontSize = 15.sp)
    }
}

6 Usages
@Composable
fun NapovedVrstica(cas: String, temp: String, emoji: String) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
```

```
            .padding(vertical = 4.dp),
        shape = RoundedCornerShape( size = 20.dp),
        colors = CardDefaults.cardColors(containerColor = Color(0x882C3E50))
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding( all = 14.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Text(cas, color = Color.White, fontSize = 16.sp)
            Row(verticalAlignment = Alignment.CenterVertically) {
                Text(temp, color = Color(0xFFE0F2FE), fontSize = 20.sp, fontWeight = FontWeight.Bold)
                Spacer(modifier = Modifier.width(12.dp))
                Text(emoji, fontSize = 26.sp)
            }
        }
    }
}

1 Usage
suspend fun pridobiVremeInPrognozo(mesto: String, kljuc: String): Pair<VremeInfo, List<PrognozaInfo>> {
    return withContext(Dispatchers.IO) {
        val urlTrenutno = "https://api.openweathermap.org/data/2.5/weather?q=$mesto,SI&appid=$kljuc&units=metric&lang=sl"
        val trenutnoJson = URL( spec = urlTrenutno).openConnection().let {
            it as HttpURLConnection
            it.inputStream.bufferedReader().readText()
        }.let { JSONObject(it) }

        val objektVreme = trenutnoJson.getJSONArray( name = "weather").getJSONObject( index = 0)
        val glavno = trenutnoJson.getJSONObject( name = "main")
        val objektVeter = trenutnoJson.getJSONObject( name = "wind")
        val objektSonce = trenutnoJson.getJSONObject( name = "sys")

        val opis = objektVreme.getString( name = "description")
        val temp = glavno.getDouble( name = "temp").toInt()
        val vlaga = glavno.getInt( name = "humidity")
        val hitrostVetra = objektVeter.getDouble( name = "speed")
```

```
        val smerVetra = objektVeter.optInt( name = "deg",  fallback = 0)
        val vzhod = objektSonce.getLong( name = "sunrise")
        val zahod = objektSonce.getLong( name = "sunset")
        val casMeritve = trenutnoJson.getLong( name = "dt")

        val trenutno = VremeInfo(
            mesto = mesto,
            cas = formatirajCas( sekunde = casMeritve),
            opis = opis,
            temperatura = "$temp °C",
            emoji = emojiVremena(opis),
            veter = "$hitrostVetra m/s, ${smerVetraString( stopinje = smerVetra)}",
            vlaga = "$vlaga %",
            soncniVzhod = formatirajCas( sekunde = vzhod),
            soncniZahod = formatirajCas( sekunde = zahod)
        )

        val urlPrognoza = "https://api.openweathermap.org/data/2.5/forecast?q=$mesto,SI&appid=$kljuc&units=metric&lang=sl"
        val prognozaJson = URL( spec = urlPrognoza).openConnection().let {
            it as HttpURLConnection
            it.inputStream.bufferedReader().readText()
        }.let { JSONObject(it) }

        val seznam = prognozaJson.getJSONArray( name = "list")
        val sedaj = System.currentTimeMillis() / 1000
        val prognoze = mutableListOf<PrognozaInfo>()
        var presteto = 0

        for (i in 0 ≤  until  < seznam.length()) {
            if (presteto >= 3) break
            val postavka = seznam.getJSONObject( index = i)
            val casUnix = postavka.getLong( name = "dt")
            if (casUnix > sedaj) {
                val tempNapoved = postavka.getJSONObject( name = "main").getDouble( name = "temp").toInt()
                val vremeNapoved = postavka.getJSONArray( name = "weather").getJSONObject( index = 0)
                val opisNapoved = vremeNapoved.getString( name = "description")
                val ura = formatirajCas( sekunde = casUnix)
                prognoze.add(
                    PrognozaInfo(
```

```
                    cas = ura,
                    temperatura = "$tempNapoved °C",
                    emoji = emojiVremena(opisNapoved)
                )
            )
            presteto++
        }
    }

    Pair(trenutno, prognoze)
  }
}

4 Usages
fun formatirajCas(sekunde: Long): String {
    val datum = Date(sekunde * 1000)
    val format = SimpleDateFormat( pattern = "HH:mm", Locale.getDefault())
    return format.format( date = datum)
}

2 Usages
fun emojiVremena(opis: String): String {
    val o = opis.lowercase()
    return when {
        o.contains( other = "dež") || o.contains( other = "rain") -> "🌧️"
        o.contains( other = "sneg") || o.contains( other = "snow") -> "❄️"
        o.contains( other = "megla") || o.contains( other = "fog") || o.contains( other = "mist") -> "🌫️"
        o.contains( other = "obla") || o.contains( other = "cloud") -> "☁️"
        o.contains( other = "jasno") || o.contains( other = "clear") -> "☀️"
        else -> "🌤️"
    }
}

1 Usage
fun smerVetraString(stopinje: Int): String {
    return when (stopinje) {
        in 0 ≤ .. ≤ 22 -> "S"
        in 23 ≤ .. ≤ 67 -> "SV"
        in 68 ≤ .. ≤ 112 -> "V"
        in 113 ≤ .. ≤ 157 -> "JV"
        in 158 ≤ .. ≤ 202 -> "J"
        in 203 ≤ .. ≤ 247 -> "JZ"
        in 248 ≤ .. ≤ 292 -> "Z"
        in 293 ≤ .. ≤ 337 -> "SZ"
        else -> "S"
    }
}
```

7_EMULATOR


4:55
Vreme
Ljubljana
Ljubljana, 16:52
Delno oblačno
27 °C
Hitrost vetra
1.34 m/s, Z
Vlažnost
52 %
Sončni vzhod
05:10
Sončni zahod
20:54
Napoved
Čez 1 uro
27 °C
Čez 2 uri
25 °C
51%